Min Cost to Cut Stick

This is one of the **hardest Partition DP problems** because the recursion doesn't feel natural at first.

The biggest mistake people make is thinking:

> "Should I cut from left to right?"

No. The recursion is **not traversing the stick**. It is **choosing which cut to perform FIRST** inside a segment.

Once you understand this, the entire DP becomes exactly like **Matrix Chain Multiplication (MCM)**.

Step 1. Understand the problem

Suppose

```
Stick Length = 9
Cuts = [2,4,7]
```

Initially

```
0-----------------------9
```

Need to cut at

```
2
4
7
```

Every time you cut a stick,

Cost = current stick length

NOT the original stick length.

Example

If first cut is at 4

```
0----------4------------9
Cost = 9
```

Now two sticks

```
0----4
```

and

```
4------9
```

Now each side is solved independently.

Step 2. Why Greedy doesn't work?

Suppose first cut at 2

```
Cost = 9
0---2------------------9
```

Remaining

```
Left : nothing
Right : cuts = 4,7
```

Total cost may become

```
9 + 7 + 5
```

Suppose first cut at 7

```
Cost = 9
0--------------7--------9
```

Remaining

```
Left : cuts 2,4
Right : nothing
```

Different answer.

Suppose first cut at 4

Again different answer.

So we must try

every possible first cut.

This is where DP starts.

Step 3. Add boundaries

Very important step.

Insert

```
0
```

and

```
n
```

For our example

```
cuts = [2,4,7]
↓
cuts = [0,2,4,7,9]
```

Now every segment is represented by two indices.

Step 4. What is our state?

Suppose recursion is

```
solve(i,j)
```

What does this mean?

It means

> Minimum cost to perform all cuts between

```
cuts[i-1]
and
cuts[j+1]
```

Notice

NOT

between

```
cuts[i]
```

and

```
cuts[j]
```

This confuses everyone.

Example

```
cuts
0 2 4 7 9
```

Indices

```
0 1 2 3 4
```

Suppose

```
solve(2,3)
```

Means

Cuts to perform are

```
4
7
```

Current stick is

```
2-------------9
```

because

```
left boundary
cuts[i-1]
= cuts[1]
=2
right boundary
cuts[j+1]
= cuts[4]
=9
```

Current length

```
9-2=7
```

So every first cut inside costs

```
7
```

Step 5. Which cut should be first?

Suppose

```
solve(2,3)
```

Available cuts

```
4
7
```

We try

First cut = 4

or

First cut =7

Exactly like MCM.

Case 1

First cut =4

```
2------4-----------9
```

Cost now

```
9-2=7
```

After cutting,

left becomes

```
2----4
```

Right becomes

```
4--------9
```

Need recursion

```
solve(2,1)
solve(3,3)
```

Notice

Left contains no cuts.

Right contains

```
7
```

Total

```
7
+
solve(2,1)
+
solve(3,3)
```

Case 2

First cut =7

```
2-----------7------9
```

Current cost

```
7
```

Now

Left

contains

```
4
```

Right

contains nothing.

So

```
7
+
solve(2,2)
+
solve(4,3)
```

Take minimum.

Step 6. Why left is

```
solve(i,k-1)
```

Suppose first cut chosen is

```
k
```

Everything before it

belongs to left stick.

Everything after it

belongs to right stick.

Exactly like

```
MCM
(A B C D)
choose C
↓
Left
A B
Right
D
```

Here

```
Cuts
2 4 7
```

Choose

```
4
```

Then

```
Left cuts
2
Right cuts
7
```

Therefore

```
solve(i,k-1)
solve(k+1,j)
```

Step 7. Dry Run of Complete Recursion

Suppose

```
cuts
0 2 4 7 9
```

Initial call

```
solve(1,3)
```

Available cuts

```
2
4
7
```

Recursion tree

```
                solve(1,3)

          /          |          \
      cut2         cut4         cut7
```

Branch 1

Choose

```
2
```

```
Cost =9
Left
```

```
solve(1,0)
Right
solve(2,3)
```

Tree

```
solve(1,3)
↓
9
+
solve(1,0)
+
solve(2,3)
```

Now

```
solve(2,3)
```

again tries

```
4
7
```

Branch 2

Choose

```
4
```

```
Cost=9
Left
solve(1,1)
Right
solve(3,3)
```

Again

Both recursively solve.

Branch 3

Choose

```
7
```

```
Cost=9
Left
solve(1,2)
Right
solve(4,3)
```

Again recurse.

Notice

The recursion is **branching on the first cut**, not moving left-to-right.

Step 8. Why cost is

```
C++
cuts[j+1] - cuts[i-1]
```

Suppose

```
solve(2,3)
```

Current stick

```
2-------------9
```

Length

```
9-2=7
```

Not

```
7-4

or

9-4
```

Entire stick being cut is

```
2→9
```

Hence

```
C++
cost = cuts[j+1]-cuts[i-1];
```

Step 9. Full Recurrence

```
solve(i,j)

=

minimum over every first cut k

{

current stick length

+

left cost

+

right cost

}
```

i.e.

```
C++
solve(i,j)=
min(
cuts[j+1]-cuts[i-1]
+
solve(i,k-1)
+
solve(k+1,j)
)
```

Step 10. Why is this exactly like MCM?

Matrix Chain Multiplication	Stick Cutting
Choose first partition	Choose first cut
Left matrices	Left cuts
Right matrices	Right cuts
Cost of multiplication	Cost of current stick
`solve(i,k)`	`solve(i,k-1)`
`solve(k+1,j)`	`solve(k+1,j)`

The only difference is how the **current cost** is calculated.

Final Intuition (Most Important)

Imagine you're holding one stick with many marked cut positions.

```
0---------2------4---------7------9
```

You ask:

> **"Which cut should I perform first?"**

Suppose you choose **4**.

The stick immediately breaks into two **independent** problems:

```
Left stick:
0------2------4

Right stick:
4------7------9
```

From this point onward:

- Nothing you do on the left affects the right.
- Nothing you do on the right affects the left.

So after making the **first cut**, the problem naturally divides into two smaller subproblems:

```
Left  -> solve(i, k-1)

Right -> solve(k+1, j)
```

Since you don't know which first cut is best, you try **every possible first cut** and take the minimum.

This "choose one partition point, then recursively solve the left and right independently" pattern is exactly what defines **Partition DP**, just like **Matrix Chain Multiplication**, **Burst Balloons**, and **Boolean Parenthesization**.

Great. This is exactly like how we converted **MCM Memoization** → **Tabulation**.

Let's first recall the memoization code.

Memoization

```
C++
class Solution {
public:
    int solve(int i, int j, vector<int>& cuts, vector<vector<int>>& dp) {

        if (i > j) return 0;

        if (dp[i][j] != -1)
            return dp[i][j];

        int mini = INT_MAX;

        for (int k = i; k <= j; k++) {

            int cost = cuts[j + 1] - cuts[i - 1]
                     + solve(i, k - 1, cuts, dp)
                     + solve(k + 1, j, cuts, dp);

            mini = min(mini, cost);
        }

        return dp[i][j] = mini;
    }

    int minCost(int n, vector<int>& cuts) {

        sort(cuts.begin(), cuts.end());

        cuts.insert(cuts.begin(), 0);
        cuts.push_back(n);

        int m = cuts.size();

        vector<vector<int>> dp(m, vector<int>(m, -1));

        return solve(1, m - 2, cuts, dp);
    }
};
```

Step 1. Identify the DP State

From recursion

```
C++
solve(i, j)
```

means

> Minimum cost to perform all cuts from index `i` to `j`.

So

```cpp
dp[i][j]
```

will store the same thing.

Step 2. Base Case

Memoization

```cpp
if(i > j)
    return 0;
```

There are no cuts left.

In tabulation we don't explicitly fill these states.

Whenever

```cpp
k == i
```

then

```cpp
dp[i][k-1]
```

becomes

```cpp
dp[i][i-1]
```

which is outside the valid range.

So while writing tabulation we check

```cpp
left = (k==i)?0:dp[i][k-1];
```

Similarly

```cpp
right = (k==j)?0:dp[k+1][j];
```

Step 3. Dependency

Our recurrence is

```cpp
dp[i][j]
depends on
dp[i][k-1]
and
dp[k+1][j]
```

Notice

Both are **smaller intervals**.

Therefore,

smaller interval must be computed first.

Exactly like MCM.

Step 4. Which order?

Suppose

```
i = 2
j = 5
```

Needs

```
dp[2][1]
dp[2][2]
dp[2][3]
dp[3][5]
dp[4][5]
...
```

These are all shorter ranges.

So compute interval length from small to large.

Step 5. Filling Order

Assume

```
Number of actual cuts = c
```

States are

```
(1,1)
(2,2)
(3,3)
```

then

```
(1,2)
(2,3)
```

then

```
(1,3)
```

Exactly the same order as MCM.

Dry Run of Filling

Suppose

```
cuts
0 2 4 7 9
```

Actual cut indices

```
1 2 3
```

DP Table

```
       j
     1   2   3
i
1
2
3
```

Length =1

Fill

```
dp[1][1]
dp[2][2]
dp[3][3]
```

Length =2

Fill

```
dp[1][2]

dp[2][3]
```

Length =3

Fill

```
dp[1][3]
```

Answer obtained.

Step 6. Loops

Length

```
C++
for(len=1; len<=c; len++)
```

Starting index

```
C++
for(i=1; i+len-1<=c; i++)
```

Ending index

```
C++
j=i+len-1;
```

Now compute answer.

Step 7. Transition

Try every first cut.

```
C++
mini = INT_MAX;

for(k=i;k<=j;k++)
{
    left =
    (k==i)?0:dp[i][k-1];

    right =
    (k==j)?0:dp[k+1][j];

    cost =
    cuts[j+1]-cuts[i-1]
    + left
    + right;

    mini=min(mini,cost);
}
```

Store

```
C++
dp[i][j]=mini;
```

Complete Tabulation Code

```
C++
class Solution {
public:
    int minCost(int n, vector<int>& cuts) {

        sort(cuts.begin(), cuts.end());

        cuts.insert(cuts.begin(), 0);
        cuts.push_back(n);

        int m = cuts.size();

        // Number of actual cuts
        int c = m - 2;
```

```
        vector<vector<int>> dp(m, vector<int>(m, 0));

        // Length of interval
        for (int len = 1; len <= c; len++) {

            // Starting index
            for (int i = 1; i + len - 1 <= c; i++) {

                int j = i + len - 1;

                int mini = INT_MAX;

                for (int k = i; k <= j; k++) {

                    int left = (k == i) ? 0 : dp[i][k - 1];
                    int right = (k == j) ? 0 : dp[k + 1][j];

                    int cost = cuts[j + 1] - cuts[i - 1]
                             + left
                             + right;

                    mini = min(mini, cost);
                }

                dp[i][j] = mini;
            }
        }

        return dp[1][c];
    }
};
```

Why does this filling order work?

Let's say we're computing:

```
dp[1][3]
```

We may need:

```
dp[1][1]
dp[1][2]
dp[2][3]
dp[3][3]
```

All of these represent **shorter intervals** than `[1,3]`.

By filling intervals in increasing order of length:

```
Length = 1
↓
Length = 2
↓
Length = 3
```

every dependency is already computed before it's needed.

A cleaner tabulation (same idea as MCM)

Many people write it using `i` decreasing and `j` increasing, just like Matrix Chain Multiplication:

```
C++
for (int i = c; i >= 1; i--) {
    for (int j = i; j <= c; j++) {

        int mini = INT_MAX;

        for (int k = i; k <= j; k++) {

            int cost = cuts[j + 1] - cuts[i - 1]
                     + (k == i ? 0 : dp[i][k - 1])
                     + (k == j ? 0 : dp[k + 1][j]);

            mini = min(mini, cost);
        }

        dp[i][j] = mini;
    }
}
```

This version is mathematically equivalent and is the one you'll often see alongside the MCM tabulation pattern. Both have **$O(c^3)$** time complexity and **$O(c^2)$** space complexity, where `c` is the number of cuts.

Let's dry run this **exact code** on a small example.

Example

```
n = 7

cuts = [1,3,4,5]
```

After sorting and adding boundaries

```
cuts = [0,1,3,4,5,7]

Index

0 1 2 3 4 5
```

Actual cuts

```
1 2 3 4
```

So

```
C++
c = 4;
```

Our DP table is

```
     j

     1   2   3   4

i

1

2

3

4
```

Initially everything is 0.

Why are we looping like this?

```
C++
for(int i=c;i>=1;i--)
```

means

```
i = 4
i = 3
i = 2
i = 1
```

For every row,

```
C++
for(int j=i;j<=c;j++)
```

means

```
j starts from i
```

So filling order becomes

```
(4,4)

(3,3)
(3,4)

(2,2)
(2,3)
(2,4)

(1,1)
(1,2)
(1,3)
(1,4)
```

Notice

every dependency is already available.

Iteration 1

i =4

Only

```
j=4
```

Compute

```
dp[4][4]
```

Only one cut

```
cut =5
```

Current stick

```
cuts[3] -> cuts[5]
4 ---------- 7
```

Length

```
7-4=3
```

Loop

```
C++
k=4
```

Cost

```
3
+
0
+
0
=
3
```

Store

```
dp[4][4]=3
```

Table

```
     1  2  3  4
1
2
3
4           3
```

Iteration 2

i=3

First

```
j=3
```

Compute

```
dp[3][3]
```

Current stick

```
3------5
```

Length

```
5-3=2
```

Only

```
k=3
```

Cost

```
2
```

Store

```
dp[3][3]=2
```

Now

```
j=4
```

Need

```
dp[3][4]
```

Current stick

```
3----------7
```

Length

```
7-3=4
```

Available cuts

```
4
5
```

First possibility

```
k=3
```

Current cost

```
4
```

Left

```
0
```

Right

```
dp[4][4]=3
```

Total

```
7
```

Second possibility

```
k=4
```

Current

```
4
```

Left

```
dp[3][3]=2
```

Right

```
0
```

Total

```
6
```

Minimum

```
6
```

Store

```
dp[3][4]=6
```

Table

```
      1  2  3  4
1
2
3         2  6
4            3
```

Iteration 3

i=2

First

```
j=2
```

Current stick

```
1------4
```

Length

```
3
```

Store

```
dp[2][2]=3
```

j=3

Need

```
dp[2][3]
```

Current stick

```
1----------5
```

Length

```
4
```

Possible cuts

```
3
4
```

k=2

Cost

```
4
+
0
+
dp[3][3]=2
```

Total

```
6
```

k=3

Cost

```
4
+
dp[2][2]=3
+
0
```

Total

```
7
```

Minimum

```
6
```

Store

```
dp[2][3]=6
```

j=4

Need

```
dp[2][4]
```

Current stick

```
1---------------7
```

Length

```
6
```

Possible cuts

```
3
4
5
```

k=2

```
6
+
0
+
dp[3][4]=6
=
12
```

k=3

```
6
+
dp[2][2]=3
+
dp[4][4]=3
=
```

```
12
```

k=4

```
6
+
dp[2][3]=6
+
0
=
12
```

Minimum

```
12
```

Store

```
dp[2][4]=12
```

Table

```
      1  2  3   4
1
2      3  6  12
3         2   6
4             3
```

Iteration 4

i=1

j=1

Current stick

```
0------3
```

Length

```
3
```

Store

```
dp[1][1]=3
```

j=2

Need

```
dp[1][2]
```

Length

```
4
```

Try

```
k=1
k=2
```

Result

```
7
```

Store

```
dp[1][2]=7
```

j=3

Need

```
dp[1][3]
```

Length

```
5
```

Already have

```
dp[1][1]
dp[2][3]
dp[1][2]
dp[3][3]
```

Everything computed.

Store answer.

---

j=4

Finally

```
dp[1][4]
```

Current stick

```
0---------------7
```

Length

```
7
```

Need

```
dp[2][4]
dp[3][4]
dp[4][4]
dp[1][3]
dp[1][2]
...
```

All already computed.

Hence answer gets stored.

Why does this order work?

The recurrence is:

```
C++
dp[i][j] =
cost
+ dp[i][k-1]
+ dp[k+1][j];
```

Suppose you're computing:

```
dp[2][4]
```

It depends on:

```
dp[2][1]
dp[2][2]
dp[2][3]
dp[3][4]
dp[4][4]
```

Notice:

- `dp[2][2]` and `dp[2][3]` are in the **same row** but **smaller columns**, so they were computed earlier because `j` increases.
- `dp[3][4]` and `dp[4][4]` are in **rows below** (`i = 3` and `i = 4`), which were already computed because `i` decreases.

That's the key intuition behind the loop order:

- `i` **goes from bottom to top** so the **right subproblem** (`dp[k+1][j]`) is ready.
- `j` **goes from left to right** so the **left subproblem** (`dp[i][k-1]`) is ready.

This is exactly the same dependency pattern used in **Matrix Chain Multiplication (MCM)** tabulation. Once you recognize this dependency graph, you'll see why this loop order is a standard template for Partition DP problems.